

# **Module 16 — SQL avancé & vues optimisées**

Bootcamp Data Analyst — From Zero to Hero | Niveau Intermédiaire · Module 16

## **Ce que tu seras capable de faire à la fin de ce module**

- Créer et utiliser une vraie base **PostgreSQL hébergée** (gratuite, zéro installation)
- Enchaîner plusieurs CTEs (Common Table Expressions) dans une même requête
- Écrire une **CTE récursive** pour parcourir une relation hiérarchique (ex : chaîne de parrainage)
- Créer une **vue** ( `CREATE VIEW` ) pour réutiliser une requête complexe ailleurs (Module 17)
- Lire un plan d'exécution ( `EXPLAIN ANALYZE` ) et comprendre l'effet d'un index

 **Durée estimée** : 2h30  **Outils** : PostgreSQL hébergé (Neon.tech, gratuit) 

**Prérequis** : Modules 06-09 (Niveau Débutant) · Module 15

## **1. Pourquoi quitter SQLiteOnline**

Depuis le Module 06, tu pratiques le SQL sur **SQLiteOnline** : parfait pour apprendre, mais une base qui vit dans ton navigateur ne survit pas d'une session à l'autre, personne d'autre ne peut s'y connecter, et ce n'est pas ce qu'utilise une vraie entreprise.

Ce module te fait passer à un **vrai PostgreSQL**, hébergé dans le cloud — le même type de base de données que tu retrouveras en poste. Et comme il est persistant et accessible depuis

n'importe quel outil, c'est cette même base que Power BI viendra interroger au **Module 17**.

## Créer ta base PostgreSQL gratuite (Neon)

1. Va sur [neon.tech](https://neon.tech) et crée un compte gratuit (aucune carte bancaire n'est demandée au moment de la rédaction de ce module — vérifie que c'est toujours le cas).
2. Crée un nouveau projet. Neon te fournit immédiatement une base PostgreSQL vide.
3. Dans le panneau de gauche, ouvre le **SQL Editor** — un éditeur SQL intégré à l'interface web, sans rien installer, avec un bouton **Run** et un bouton **Explain/Analyze** directement accessibles.

⚠ **Important à savoir** : sur le tier gratuit, ta base se **met en veille après 5 minutes d'inactivité** (ce comportement n'est pas désactivable sur le tier gratuit). La première requête après une pause peut prendre quelques secondes le temps que la base se réveille — ce n'est pas un bug, c'est normal. Patiente et relance ta requête si besoin.

## 2. Créer le schéma — un réseau de parrainage télécom

On reprend le contexte télécom (ANSUT-CI / MobiCI) des Modules 08-09, avec une nuance nouvelle : chaque client peut avoir été **parrainé** par un autre client — une relation qui va nous servir pour la CTE récursive.

Colle ce script dans le SQL Editor de Neon et clique sur **Run** :

SQL

```
CREATE TABLE clients (  
    id_client SERIAL PRIMARY KEY,  
    nom TEXT NOT NULL,  
    ville TEXT NOT NULL,  
    id_parrain INTEGER REFERENCES clients(id_client),  
    date_inscription DATE NOT NULL  
);  
  
CREATE TABLE transactions (  
    id_transaction SERIAL PRIMARY KEY,  
    id_client INTEGER NOT NULL REFERENCES clients(id_client),  
    montant NUMERIC NOT NULL,  
    type TEXT NOT NULL,  
    date_transaction DATE NOT NULL  
);  
  
INSERT INTO clients (nom, ville, id_parrain, date_inscription) VALUES  
( 'Kouassi Ama', 'Abidjan', NULL, '2023-01-10'),  
( 'Traoré Ibrahim', 'Bouaké', 1, '2023-02-15'),  
( 'Diabaté Fatou', 'Abidjan', 1, '2023-02-20'),  
( 'Koné Souleymane', 'Yamoussoukro', 2, '2023-03-05'),  
( 'Bamba Aïcha', 'San-Pédro', 2, '2023-03-18'),  
( 'N'Guessan Paul', 'Korhogo', 3, '2023-04-02'),  
( 'Ouattara Mariam', 'Abidjan', 4, '2023-04-22'),  
( 'Yao Kouadio', 'Bouaké', NULL, '2023-01-25'),  
( 'Coulibaly Awa', 'Man', 8, '2023-02-10'),  
( 'Sanogo Moussa', 'Abidjan', 8, '2023-03-01');  
  
INSERT INTO transactions (id_client, montant, type, date_transaction) VALUES  
(1, 15000, 'depot', '2023-05-01'), (1, 5000, 'retrait', '2023-05-10'),  
(2, 25000, 'depot', '2023-05-03'), (2, 10000, 'transfert', '2023-05-15'),  
(3, 8000, 'depot', '2023-05-04'), (4, 30000, 'depot', '2023-05-06'),  
(4, 12000, 'retrait', '2023-05-20'), (5, 6000, 'depot', '2023-05-07'),  
(6, 45000, 'depot', '2023-05-08'), (7, 9000, 'transfert', '2023-05-09'),  
(8, 22000, 'depot', '2023-05-02'), (9, 7000, 'depot', '2023-05-11'),  
(10, 18000, 'depot', '2023-05-12'), (1, 20000, 'depot', '2023-06-01'),  
(3, 15000, 'transfert', '2023-06-02');
```

`id_parrain` référence la même table `clients` — c'est une **relation réflexive** : un client peut être le parrain d'un autre client.

### 3. CTEs multiples — enchaîner les étapes

Tu as vu une CTE simple au Module 09. Ici, on **enchaîne deux CTEs** : la première calcule un total par client, la seconde s'en sert pour comparer chaque client à la moyenne générale.

SQL

```
WITH totaux_client AS (  
    SELECT id_client, SUM(montant) AS total_transactions, COUNT(*) AS  
    nb_transactions  
    FROM transactions  
    GROUP BY id_client  
)  
avec_moyenne AS (  
    SELECT id_client, total_transactions, nb_transactions,  
           ROUND(AVG(total_transactions) OVER (), 0) AS moyenne_generale  
    FROM totaux_client  
)  
SELECT c.nom, a.total_transactions, a.moyenne_generale,  
       a.total_transactions - a.moyenne_generale AS ecart  
FROM avec_moyenne a  
JOIN clients c ON c.id_client = a.id_client  
ORDER BY ecart DESC;
```

La deuxième CTE (`avec_moyenne`) utilise directement la première (`totaux_client`) — chaque CTE peut réutiliser tout ce qui est défini avant elle dans le même `WITH`. C'est plus lisible qu'une seule requête imbriquée avec des sous-requêtes empilées.

#### 👉 À toi de jouer

Ajoute une troisième CTE qui ne garde que les clients dont l'écart à la moyenne est positif (au-dessus de la moyenne). Indice : tu peux référencer `avec_moyenne` dans une nouvelle CTE avec un `WHERE`, exactement comme tu le ferais sur une table normale.

## 4. CTE réursive — remonter une chaîne de parrainage

Une **CTE réursive** (`WITH RECURSIVE`) se construit en deux parties : une **ancree** (le point de départ) et une **réursion** (qui se rappelle elle-même jusqu'à épuisement des résultats). C'est l'outil de référence pour parcourir une hiérarchie : organigramme, arborescence de catégories, ou ici, une chaîne de parrainage.

SQL

```
WITH RECURSIVE chaine_parrainage AS (  
  -- Ancree : les clients sans parrain (racines de l'arbre)  
  SELECT id_client, nom, id_parrain, 0 AS niveau  
  FROM clients  
  WHERE id_parrain IS NULL  
  
  UNION ALL  
  
  -- Réursion : chaque client dont le parrain est déjà dans la chaîne  
  SELECT c.id_client, c.nom, c.id_parrain, cp.niveau + 1  
  FROM clients c  
  JOIN chaine_parrainage cp ON c.id_parrain = cp.id_client  
)  
SELECT niveau, nom, id_parrain  
FROM chaine_parrainage  
ORDER BY niveau, nom;
```

Comment la lire :

1. L'**ancree** sélectionne les clients sans parrain (`id_parrain IS NULL`) — niveau 0, les racines.
2. La **réursion** rejoint `clients` à `chaine_parrainage` elle-même : à chaque tour, elle trouve les clients dont le parrain vient d'être ajouté au tour précédent, et incrémente `niveau`.
3. La réursion s'arrête automatiquement quand elle ne trouve plus aucun nouveau client à ajouter (plus personne dont le parrain est dans la chaîne mais qui n'y est pas encore).

Tu dois obtenir une chaîne sur 4 niveaux : 2 clients au niveau 0 (sans parrain), leurs filleuls directs au niveau 1, les filleuls de leurs filleuls au niveau 2, et ainsi de suite.

⚠ **Piège classique** : si tes données contiennent un cycle (A parraine B, B parraine A — normalement impossible ici grâce à la logique métier, mais toujours possible en cas d'erreur de saisie), une CTE récursive **tourne à l'infini**. En production, on ajoute une limite de sécurité, par exemple `WHERE niveau < 20` dans la partie récursive, pour se protéger de ce cas.

## 5. Créer une vue — pour la réutiliser au Module 17

Une **vue** (`CREATE VIEW`) enregistre une requête sous un nom, comme si c'était une table — sans dupliquer les données. Chaque fois qu'on interroge la vue, la requête sous-jacente est réexécutée sur les données à jour.

SQL

```
CREATE VIEW vue_resume_clients AS
SELECT c.id_client, c.nom, c.ville, c.date_inscription,
       COUNT(t.id_transaction) AS nb_transactions,
       COALESCE(SUM(t.montant), 0) AS total_transactions
FROM clients c
LEFT JOIN transactions t ON t.id_client = c.id_client
GROUP BY c.id_client, c.nom, c.ville, c.date_inscription;
```

Une fois créée, tu l'interroges exactement comme une table :

SQL

```
SELECT * FROM vue_resume_clients ORDER BY total_transactions DESC LIMIT 5;
```

💡 **Pourquoi une vue plutôt qu'une simple requête sauvegardée ?** Parce qu'un outil externe comme **Power BI** (Module 17) peut se connecter directement à `vue_resume_clients` comme s'il s'agissait d'une table — sans avoir besoin de connaître ni de recopier toute la logique SQL derrière. Tu prépares la donnée une fois, ici ; elle est ensuite réutilisable partout.

🔗 Le `LEFT JOIN` + `COALESCE` : remarque qu'on utilise `LEFT JOIN` (pas `INNER JOIN`) pour garder les clients même s'ils n'ont **aucune** transaction, et `COALESCE(..., 0)` pour afficher `0` plutôt qu'une valeur vide dans ce cas — un réflexe déjà vu au Module 08 pour les anti-jointures.

## 6. Lire un plan d'exécution — `EXPLAIN ANALYZE`

`EXPLAIN ANALYZE` exécute réellement ta requête et affiche **comment** PostgreSQL s'y est pris — utile pour comprendre pourquoi une requête est lente sur un vrai volume de données (le SQL Editor de Neon a un bouton dédié pour ça).

### Avant un index

```
SQL
EXPLAIN ANALYZE
SELECT * FROM clients WHERE ville = 'Abidjan';
```

Tu dois voir apparaître un `Seq Scan on clients` ("Seq" pour séquentiel) : PostgreSQL lit **toute la table**, ligne par ligne, pour trouver celles qui correspondent à `ville = 'Abidjan'`. Sur 10 lignes, c'est instantané — sur 10 millions de lignes, ce serait très lent.

### Créer un index

```
SQL
CREATE INDEX idx_clients_ville ON clients(ville);
```

Un index sur `ville` fonctionne comme l'index d'un livre : au lieu de parcourir toutes les pages, PostgreSQL sait directement où chercher.

## Après l'index

SQL

```
EXPLAIN ANALYZE
```

```
SELECT * FROM clients WHERE ville = 'Abidjan';
```

💡 Sur une aussi petite table (10 lignes), PostgreSQL peut choisir de garder un **Seq Scan** même après la création de l'index — parce que pour un si petit volume, parcourir toute la table est déjà plus rapide que consulter l'index. C'est une vraie décision d'optimiseur, pas un bug : PostgreSQL choisit toujours le plan **le moins coûteux estimé**, pas nécessairement celui qui "a l'air" le plus sophistiqué. L'écart entre **Seq Scan** et **Index Scan** ne devient visible qu'à partir de quelques milliers de lignes — retiens la logique, pas le comportement exact sur ce jeu de données miniature.

Les 3 informations à repérer dans un plan d'exécution :

| ÉLÉMENT                              | CE QUE ÇA TE DIT                                           |
|--------------------------------------|------------------------------------------------------------|
| <b>Seq Scan</b> vs <b>Index Scan</b> | Table entière parcourue, ou recherche ciblée via un index  |
| <b>cost=X..Y</b>                     | Estimation du coût (unité arbitraire), avant exécution     |
| <b>actual time=X..Y</b>              | Temps réellement mesuré, en millisecondes, après exécution |

## ✅ Résumé du module



| CONCEPT                                       | CE QU'IL FAUT RETENIR                                                                                                                                 |
|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| PostgreSQL hébergé                            | Un vrai serveur, persistant, accessible par plusieurs outils — contrairement à SQLiteOnline                                                           |
| Scale-to-zero (tier gratuit Neon)             | La base se met en veille après 5 min d'inactivité, non désactivable gratuitement — la première requête après une pause peut prendre quelques secondes |
| CTEs multiples                                | Chaque <code>WITH ... AS (...)</code> peut réutiliser les CTEs définies avant elle dans la même requête                                               |
| CTE récursive ( <code>WITH RECURSIVE</code> ) | Ancre (point de départ) + récursion (se rappelle jusqu'à épuisement) — pour parcourir une hiérarchie                                                  |
| Risque des CTE récursives                     | Un cycle dans les données peut créer une boucle infinie — prévoir une limite de sécurité                                                              |
| <code>CREATE VIEW</code>                      | Sauvegarde une requête sous un nom, réutilisable comme une table par d'autres outils (Power BI au M17)                                                |
| <code>EXPLAIN ANALYZE</code>                  | Montre le plan d'exécution réel : <code>Seq Scan</code> (table entière) vs <code>Index Scan</code> (recherche ciblée)                                 |
| Index                                         | Accélère les recherches sur une colonne précise — l'effet ne se voit que sur des volumes suffisants                                                   |

## Quiz — Vérifie ta compréhension

Q1. Dans une CTE récursive, à quoi sert la partie "ancree" ?

- a) Elle définit la condition d'arrêt de la récursion
- b) Elle définit le point de départ (les premières lignes, avant toute récursion)
- c) Elle supprime les doublons

👉 Voir la réponse

✓ **b)** — L'ancre est la requête initiale (avant le `UNION ALL`), qui fournit le point de départ. La récursion s'arrête automatiquement quand elle ne trouve plus de nouvelle ligne à ajouter — il n'y a pas besoin d'écrire une condition d'arrêt explicite dans le cas général.

**Q2.** Que risques-tu si tes données contiennent un cycle (A parraine B, B parraine A) dans une CTE récursive ?

- a) Une erreur de syntaxe immédiate
- b) Une boucle infinie, sauf si tu ajoutes une limite de sécurité (ex : `WHERE niveau < 20`)
- c) Rien, PostgreSQL détecte et ignore automatiquement les cycles

👉 Voir la réponse

✓ **b)** — PostgreSQL ne détecte pas automatiquement les cycles dans une CTE récursive standard ; il continuera à générer des lignes indéfiniment. D'où l'importance d'ajouter une limite de profondeur en production.

**Q3.** Pourquoi créer une vue (`CREATE VIEW`) plutôt que de recopier la requête à chaque fois ?

- a) Une vue est plus rapide qu'une requête normale, dans tous les cas
- b) Une vue peut être interrogée comme une table par d'autres outils (Power BI...), sans dupliquer la logique SQL
- c) Une vue stocke une copie figée des données au moment de sa création

👉 Voir la réponse

✓ **b)** — Une vue centralise la logique de requête à un seul endroit et peut être consommée par n'importe quel outil qui sait lire une base PostgreSQL. C'est faux pour c) : une vue standard (non matérialisée) réexécute la requête à chaque appel, elle reflète toujours les données actuelles — pas une copie figée.

**Q4.** Sur une table de seulement 10 lignes, **EXPLAIN ANALYZE** montre un **Seq Scan** même après avoir créé un index sur la colonne filtrée. Que dois-tu en conclure ?

- a) L'index n'a pas été créé correctement
- b) PostgreSQL a estimé qu'un parcours complet était moins coûteux qu'utiliser l'index sur un si petit volume
- c) Il faut recréer l'index avec une syntaxe différente

 [Voir la réponse](#)

✓ **b)** — Sur une très petite table, parcourir toutes les lignes est souvent plus rapide que consulter un index. PostgreSQL choisit toujours le plan qu'il estime le moins coûteux — pas nécessairement celui qui utilise l'index disponible. L'avantage d'un index ne devient net qu'à partir d'un volume de données suffisant.

## **Module suivant**

Ta base PostgreSQL est prête, avec ses CTEs, sa vue et son index. Direction le **Module 17**, où tu connectes Power BI à cette même base pour construire un dashboard.

→ **Module 17 — Power BI connecté à SQL**